

JAVA INTERVIEW QUESTION BANK

Topic-wise Questions + Detailed Interview-Ready Answers

Reorganized from the provided Question Bank

Format: detailed explanation • why it matters • where it is used • comparisons • code • interview notes

Contents

- 1. Java Fundamentals & Object Model
- 2. OOP, Interfaces, Immutability & Design
- 3. Strings
- 4. Collections, Hashing & Generics
- 5. Functional Interfaces & Streams
- 6. Exceptions & Resource Management
- 7. Serialization, Copying & Object Lifecycle
- 8. Multithreading & Concurrency
- 9. JVM Memory Model, OOM & Garbage Collection
- 10. Modern Java: 8 / 11 / 17 / 21 / 25
- 11. Annotations
- 12. Testing, Mockito & Dependency Injection
- 13. Reactive Programming
- 14. Stream API Coding Questions

NOTE: Duplicate questions from the original mixed list are consolidated under the correct topic so you do not study the same concept twice. Every unique question/topic from the source is represented.

1. Java Fundamentals & Object Model

Q1. Why is the main method static?

Interview-ready explanation: The Java launcher needs a known entry point that it can call before any application object exists. Declaring main as static makes it belong to the class, so the JVM can invoke it directly after loading the class instead of first deciding how to construct an instance.

Why / where it is used: This matters for command-line applications, service bootstrapping and interview questions about static members. The method itself is only the starting hook; real application logic is usually delegated to objects/services after startup.

In the traditional Java launcher model, the JVM must be able to invoke the entry point without first creating an object of the class. Therefore the standard entry point is static: `public static void main(String[] args)`.

- public: launcher can access it.
- static: no instance is required.
- void: no return value is expected by the JVM.
- String[] args: receives command-line arguments.

NOTE: Java 25 also supports compact source files and instance main methods in simplified programs, but the traditional static main signature remains standard interview knowledge.

Q2. Is Java pass-by-value?

Interview-ready explanation: Java is always pass-by-value. For a primitive, the copied value is the primitive itself; for an object, the copied value is the reference. Both caller and callee may therefore refer to the same object, but reassigning the parameter inside the method never changes the caller's variable.

Why / where it is used: This distinction explains why a method can mutate an object's fields but cannot 'replace' the caller's object reference. It is important when debugging side effects, API design and interview questions that incorrectly describe Java as pass-by-reference.

Yes. Java is always pass-by-value. For primitives, the primitive value is copied. For objects, the value being copied is the object reference.

```
static void change(Person p) {  
    p.name = "Bob";    // Mutates the same object  
    p = new Person();  // Only changes local copy of reference  
}
```

INTERVIEW TIP: Java does not pass objects by reference. It passes a copy of the reference by value.

Q3. How many ways can a Java object be created?

Interview-ready explanation: Objects can be created directly with `new`, reflectively through constructors, by cloning when clone semantics are provided, or by deserialization. Framework factories may hide the mechanism, but they ultimately use one of these object-creation paths.

Why / where it is used: Know this because dependency-injection, ORM and serialization frameworks often instantiate classes without an obvious `new` expression in your business code.

A practical modern answer is four core mechanisms. Some interview lists say five by splitting reflection APIs into separate items.

- new keyword: new Employee().
- Reflection: Constructor.newInstance() (Class.newInstance() is deprecated).
- Cloning: clone() when Cloneable/clone semantics are provided.
- Deserialization: reconstructing an object from serialized form.
- Factory/framework APIs may create objects, but internally they eventually use one of these mechanisms.

Q4. What is the superclass of all Java classes? Does a class extend Object plus another class?

Interview-ready explanation: java.lang.Object is the root class of the normal Java class hierarchy. A class can extend only one class directly; if B extends A, then B inherits Object indirectly through A rather than extending both A and Object separately.

Why / where it is used: Object defines behavior every ordinary object can participate in, such as equals/hashCode/toString and monitor methods wait/notify/notifyAll. Understanding this also clarifies Java's single class inheritance model.

java.lang.Object is the root of the Java class hierarchy. If class B extends A, and A implicitly extends Object, then B indirectly inherits Object; B is not extending two classes.

- Common Object methods: equals(), hashCode(), toString(), getClass(), clone(), wait(), notify(), notifyAll().
- finalize() also comes from Object but finalization is deprecated for removal and should not be used.

```
class A { }    // A implicitly extends Object
class B extends A { } // B -> A -> Object
```

Q5. == vs equals() vs hashCode()

Interview-ready explanation: For objects, == asks whether two references point to the same object. equals() expresses logical equality when a class overrides it, while hashCode() provides the integer hash used by hash-based collections. The three concepts are related but answer different questions.

Why / where it is used: This is critical when using HashMap or HashSet and when building value/domain objects. If logical equality is customized, equals() and hashCode() must be consistent so lookup, duplicate detection and removal work correctly.

- If a.equals(b) is true, a.hashCode() must equal b.hashCode().
- If hash codes are equal, objects do not have to be equal; collisions are allowed.
- If equals() is overridden, hashCode() should also be overridden.

Item	Meaning	Typical use
==	Primitive value comparison; reference identity for objects	Check same reference / primitive equality
equals()	Logical/content equality if overridden	String, value objects, domain equality
hashCode()	Integer hash used by hash-based collections	HashMap, HashSet bucket selection

INTERVIEW TIP: HashMap uses hashCode() to narrow the bucket and equals() to find the exact key within that bucket.

Q6. final vs finally vs finalize()

Interview-ready explanation: final is a language modifier, finally is part of exception handling, and finalize() was an old Object cleanup hook. They are unrelated despite similar names. final can protect a reference from reassignment, a method from overriding or a class from inheritance; finally is normally used for cleanup logic.

Why / where it is used: In modern Java, resource cleanup should usually use try-with-resources rather than finalize(), because finalization is unpredictable and deprecated for removal.

Keyword / Method	Meaning
final	Prevents reassignment of a variable; prevents method overriding; prevents class inheritance.
finally	Block associated with try/catch that normally runs for cleanup whether an exception occurs or not.
finalize()	Old GC finalization hook. Deprecated for removal; unpredictable and should not be used.

Q7. transient vs volatile vs static keyword

Interview-ready explanation: transient controls Java serialization, volatile controls cross-thread visibility and ordering for a field, and static makes a member belong to the class rather than each instance. They solve three completely different problems.

Why / where it is used: Use transient for non-serializable/sensitive derived state, volatile for simple shared state such as stop flags or safely published references, and static for class-wide constants, factories or shared class behavior.

Keyword	Purpose	Key point
transient	Serialization	Field is skipped by default Java serialization.
volatile	Concurrency / visibility	Reads/writes go through JMM visibility guarantees; does not make compound operations such as count++ atomic.
static	Class-level member	One member belongs to the class rather than each object instance.

Q8. What is Optional?

Interview-ready explanation: Optional<T> models 'a value may or may not be present' explicitly instead of returning null. It encourages callers to handle absence through map/filter/orElse/orElseGet/orElseThrow rather than scattering null checks.

Why / where it is used: It is most useful as a method return type in APIs such as findById(). It is not a general replacement for every nullable field or parameter, and Optional.get() without checking presence defeats most of its benefit.

Optional<T> is a container that represents either a present non-null value or no value. It is mainly used as a return type to make absence explicit and reduce null-handling mistakes.

- Useful methods: of(), ofNullable(), empty(), map(), flatMap(), filter(), orElse(), orElseGet(), orElseThrow().
- Avoid using Optional as a field/parameter by default; it is primarily designed for return values.

```
Optional<User> user = findUser(id);  
String name = user.map(User::getName).orElse("Unknown");
```

Q9. Iterate over a String without using predefined methods

Interview-ready explanation: The usual intent of this question is to avoid convenience conversions such as toCharArray() or streams. The straightforward solution is indexed traversal using length() and charAt(). Strictly speaking, a String cannot be inspected without using some String/API operation at all.

Why / where it is used: This tests basic control flow and understanding of String indexing; in production, choose the clearest API unless an interview constraint forbids it.

If “no predefined methods” is interpreted literally, Java provides no way to access String characters without using some String/API operation. The usual interview intent is “without toCharArray()/streams,” so use an index loop with charAt().

```
String s = "Java";  
for (int i = 0; i < s.length(); i++) {  
    System.out.println(s.charAt(i));  
}
```

2. OOP, Interfaces, Immutability & Design

[VERY IMPORTANT] Q10. Overloading vs overriding

Interview-ready explanation: Overloading means the compiler chooses among methods with the same name but different parameter lists. Overriding means a subclass supplies a new implementation of an inherited instance method, and runtime dispatch chooses the implementation based on the actual object.

Why / where it is used: Overloading is useful for ergonomic APIs with different input forms; overriding is the basis of runtime polymorphism, framework callbacks and extensible object-oriented designs.

Overloading	Overriding
Same method name, different parameter list	Subclass provides new implementation of inherited instance method
Usually compile-time polymorphism	Runtime polymorphism
Return type alone cannot overload	Covariant return type is allowed
Can occur in same class	Requires inheritance
static/final/private methods can be overloaded	final/private cannot be overridden; static is hidden

Q11. What happens when a static method is “overridden”? Method hiding

Interview-ready explanation: Static methods belong to the class and are resolved using the compile-time reference/class, so they are not dynamically dispatched. A same-signature static method in a subclass hides the parent method rather than overriding it.

Why / where it is used: This matters because calling a static method through an instance can visually resemble polymorphism even though the selected method is determined by the reference type.

Static methods are resolved by the reference/class type, not dynamically by the runtime object. A subclass may declare a static method with the same signature, but that is method hiding, not overriding.

```
class A { static void show() { System.out.println("A"); } }  
class B extends A { static void show() { System.out.println("B"); } }
```

```
A ref = new B();  
ref.show(); // A
```

Q12. Overriding rules with exceptions, static and final methods

Interview-ready explanation: Java places strict rules on overriding so a subclass remains substitutable for the parent contract. An override cannot reduce visibility or add a broader checked exception; final methods cannot be overridden, private methods are not inherited, and static methods are hidden rather than overridden.

Why / where it is used: These rules are especially important in framework base classes and public APIs where callers compile against the parent type.

- Checked exception: overriding method may throw the same, a narrower checked exception, or none; it cannot throw a broader checked exception.
- Unchecked exception: no such narrowing restriction.

- static method: hidden, not overridden.
- final method: cannot be overridden.
- private method: not inherited, so it is not overridden.
- Access level cannot be made more restrictive in the overriding method.

Q13. Covariant return types

Interview-ready explanation: A covariant return type allows an overriding method to return a more specific reference type than the parent method. The parameter list still has to match; only the reference return type becomes narrower.

Why / where it is used: It improves API usability because callers of the subclass can receive a specific type without an extra cast while preserving the parent contract.

When overriding a method, the child method may return a subtype of the parent method's reference return type.

```
class Parent { Number getValue() { return 1; } }
class Child extends Parent {
    @Override Integer getValue() { return 1; } // Covariant return
}
```

Q14. Abstract class vs interface

Interview-ready explanation: An abstract class is a partially implemented base class that can hold instance state, constructors and both abstract/concrete methods. An interface primarily defines a contract/capability and supports multiple implementation; modern interfaces can also contain default, static and private helper methods.

Why / where it is used: Prefer an interface when unrelated classes should promise the same behavior. Prefer an abstract class when subclasses genuinely share state, lifecycle or common implementation that belongs in one inheritance hierarchy.

Abstract class	Interface
Can have instance fields/state	Fields are implicitly public static final constants
Can have constructors	No normal constructors
Abstract + concrete methods	Abstract methods plus default/static methods; private helper methods are allowed in modern Java
Single class inheritance	A class can implement multiple interfaces
Use for shared state + common base behavior	Use for capability/contract across unrelated classes

INTERVIEW TIP: Choose an interface for a contract/capability; choose an abstract class when subclasses genuinely share state or implementation.

Q15. Default and static methods in interfaces

Interview-ready explanation: Default methods let an interface add inheritable behavior without forcing every existing implementation to change. Static interface methods provide utility behavior scoped to the interface, and Java 9+ private methods let default/static methods share internal helper logic.

Why / where it is used: These features are heavily used for API evolution, for example the Java Collections interfaces gained default methods such as `removeIf()` and stream-related conveniences without breaking all implementations.

- default method: has an implementation and is inherited by implementing classes unless overridden.
- static method: belongs to the interface and is called with `InterfaceName.method()`; it is not inherited as an instance method.
- Java 9+ interfaces can also have private helper methods to share implementation between default/static methods.

```
interface Payment {  
    default void printReceipt() { System.out.println("Receipt"); }  
    static boolean valid(int amount) { return amount > 0; }  
}
```

Q16. Diamond problem with two interface default methods having the same signature

Interview-ready explanation: If a class implements two interfaces that provide the same default method signature, Java cannot choose one automatically. The class must override the method and either provide its own implementation or explicitly invoke `InterfaceName.super.method()`.

Why / where it is used: This is Java's controlled form of the diamond problem: multiple interface inheritance is allowed, but ambiguous inherited behavior must be resolved explicitly.

The implementing class gets an ambiguous inherited default method, so it must explicitly override the method and resolve the conflict.

```
interface I1 { default void hello() { System.out.println("I1"); } }  
interface I2 { default void hello() { System.out.println("I2"); } }  
  
class Demo implements I1, I2 {  
    @Override  
    public void hello() {  
        I1.super.hello(); // Explicit choice  
    }  
}
```

INTERVIEW TIP: Class methods win over interface defaults; otherwise conflicting defaults must be resolved explicitly.

Q17. Composition vs inheritance

Interview-ready explanation: Inheritance models an 'is-a' relationship and exposes a subclass to the parent's contract and implementation. Composition models a 'has-a' relationship by storing collaborators as fields. Composition is usually more flexible because behavior can be replaced without changing the inheritance hierarchy.

Why / where it is used: Use inheritance when the subtype truly satisfies the parent contract. Use composition for services, strategies, repositories and most reusable business behavior where you want lower coupling and easier testing.

Composition	Inheritance
HAS-A relationship	IS-A relationship
Object contains/delegates to another object	Subclass extends parent
Loose coupling, flexible behavior	Tighter coupling

Composition	Inheritance
Preferred for reuse when no true subtype relation exists	Use when subtype substitutability is valid

Q18. Association vs Aggregation vs Composition

Interview-ready explanation: Association is the broadest relationship: two objects know or use each other. Aggregation is a weaker whole-part relationship where parts can exist independently, while composition is strong ownership where the part's lifecycle is tied to the whole.

Why / where it is used: These distinctions help communicate domain design and ownership, especially in UML, entity modeling and deciding who is responsible for creating/destroying child objects.

Relationship	Meaning	Lifecycle
Association	General relationship between independent objects	Independent
Aggregation	Weak HAS-A ownership	Part can outlive whole
Composition	Strong HAS-A ownership	Part lifecycle tied to whole

```
// Association: Doctor <-> Patient
// Aggregation: Department has Teachers
// Composition: House has Rooms
```

Q19. What is a Singleton class?

Interview-ready explanation: A Singleton restricts controlled access to one logical instance of a class. The usual implementation hides construction and exposes a class-level access point; however, a singleton is still global mutable state if it holds changing data.

Why / where it is used: Use it sparingly for truly process-wide services/configuration. In modern applications dependency-injection containers often manage a single scoped instance more cleanly than hand-written Singleton code.

A Singleton restricts a class to one shared instance and provides a global access point to that instance.

```
public final class Singleton {
    private Singleton() { }
    private static final Singleton INSTANCE = new Singleton();
    public static Singleton getInstance() { return INSTANCE; }
}
```

Q20. Singleton in a multithreaded environment: double-checked locking

Interview-ready explanation: Double-checked locking reduces synchronization after a lazy Singleton has been initialized: it checks the field, synchronizes only when needed, then checks again inside the lock. The instance must be volatile so construction is safely published and reads cannot observe an incompletely initialized object.

Why / where it is used: This is an important Java Memory Model example. In most code, eager initialization, the initialization-on-demand holder idiom, enum singleton, or DI-managed singleton is simpler.

Double-checked locking avoids synchronizing every `getInstance()` call after initialization. The instance must be volatile so publication is safe under the Java Memory Model.

```
public final class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

INTERVIEW TIP: An enum singleton is often the simplest robust singleton in Java, especially when serialization is involved.

Q21. If a Singleton implements Serializable, what is the impact?

Interview-ready explanation: Default Java deserialization reconstructs a new object, so serializing and later deserializing a Singleton can create a second instance and violate the one-instance guarantee.

Why / where it is used: This matters when old-style Java serialization is used for cached/session data or persisted configuration. A singleton that is never serialized does not have this specific problem.

Deserialization normally creates a new object instance, which can break the Singleton guarantee even though the serialized object originally came from the singleton instance.

Q22. How do you resolve the Singleton serialization issue?

Interview-ready explanation: A `readResolve()` method can replace the deserialized object with the already-existing Singleton instance. During deserialization the JVM calls this hook and uses the returned object as the resolved result.

Why / where it is used: An enum Singleton is simpler and inherently resistant to this serialization issue, which is one reason it is often recommended for pure singleton semantics.

Implement `readResolve()` so deserialization returns the existing singleton instance instead of the newly reconstructed instance.

```
private Object readResolve() {
    return getInstance();
}
```

NOTE: An enum singleton gets serialization safety automatically.

[VERY IMPORTANT] Q23. Why are Strings immutable?

Interview-ready explanation: String is immutable: once a String object is created, its character content cannot be changed. Operations such as `concat()`, `replace()` or `toUpperCase()` return another String rather than modifying the original object.

Why / where it is used: Immutability makes the String Pool safe, improves security for paths/URLs/class names, allows Strings to be shared safely across threads, and makes their hash code stable when used as HashMap keys.

- String pool safety: shared literals cannot be changed by one reference and corrupt another.
- Security: Strings are used for class names, paths, URLs and other sensitive values.
- Thread safety: immutable objects can be shared safely.
- Hashing: String hash codes can be cached; Strings are reliable HashMap keys.
- Predictability: methods can safely reuse/share String instances.

Q24. How do you design an immutable class?

Interview-ready explanation: An immutable class exposes no operation that can change its observable state after construction. Making the class final, fields private/final and providing no setters is only part of the job; mutable objects passed in or returned must also be defensively copied.

Why / where it is used: Immutable objects are easier to reason about, naturally thread-safe for sharing and reliable as map keys/value objects. They are common in configuration, money/date/value types and concurrent code.

- Make the class final, or otherwise prevent unsafe subclassing.
- Make fields private and final.
- Initialize all state in the constructor.
- Provide no setters or mutating methods.
- Defensively copy mutable inputs.
- Return defensive copies (or immutable views) of mutable internal state.

Q25. Create an immutable Employee class containing Address as an instance variable

Interview-ready explanation: If an immutable Employee contains a mutable Address, merely making the Address field final is not enough: final prevents reassigning the reference, not mutating the Address object. The constructor must copy the incoming Address and the getter must return a copy (or an immutable representation).

Why / where it is used: This is a classic interview test of defensive copying and the difference between reference immutability and deep/object immutability.

Because Address is mutable, Employee must make defensive copies when it receives and returns Address.

```
final class Address {
    private String city;
    Address(String city) { this.city = city; }
    String getCity() { return city; }
    void setCity(String city) { this.city = city; }
}

public final class Employee {
    private final int id;
    private final String name;
    private final Address address;

    public Employee(int id, String name, Address address) {
        this.id = id;
        this.name = name;
        this.address = new Address(address.getCity()); // Defensive copy
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public Address getAddress() {
        return new Address(address.getCity()); // Defensive copy
    }
}
```

Q26. What is a Marker interface?

Interview-ready explanation: A marker interface has no behavior methods; implementing it marks a type as belonging to a category that runtime/framework code understands. Serializable and Cloneable are classic examples.

Why / where it is used: Marker interfaces can participate in the type system, so APIs can require the marker as a parameter bound. Annotations are often preferred when only metadata is needed and no type relationship is required.

A marker interface has no methods; implementing it marks a class as having a capability or meaning recognized by Java/framework code.

- Examples: Serializable, Cloneable.
- Modern designs often use annotations when behavior does not require type-system participation.

3. Strings

Q27. String pool and String immutability

Interview-ready explanation: String literals are interned in the String Pool. When two identical literals are used, the JVM can let both references point to the same pooled String because the object cannot later be mutated.

Why / where it is used: Use literals for normal fixed text. `new String("abc")` deliberately creates another heap object and is rarely needed; `equals()` compares content while `==` only checks whether the references are identical.

String literals are interned in the String Pool. Equal literals can share one String object because String is immutable.

```
String a = "Java";
String b = "Java";
System.out.println(a == b); // true: same pooled literal

String c = new String("Java");
System.out.println(a == c);    // false
System.out.println(a.equals(c)); // true
```

[VERY IMPORTANT] Q28. String vs StringBuilder vs StringBuffer

Interview-ready explanation: String is immutable; StringBuilder and StringBuffer are mutable character sequences. StringBuilder is not synchronized and is normally the fastest choice for repeated changes in one thread, while StringBuffer synchronizes its methods for shared mutable access.

Why / where it is used: Use String for stable text and API values, StringBuilder for loops/string assembly, and StringBuffer only when you specifically need its built-in synchronization on one shared buffer.

Type	Mutable?	Thread-safe?	Best use
String	No	Safe to share because immutable	Text that is not repeatedly modified
StringBuilder	Yes	No	Repeated string building in normal single-threaded code
StringBuffer	Yes	Its methods are synchronized	Shared mutable character buffer when that synchronization is specifically needed

INTERVIEW TIP: For repeated concatenation in loops, prefer StringBuilder. Simple + concatenations are fine when readability is best and the compiler can optimize them.

4. Collections, Hashing & Generics

[VERY IMPORTANT] Q29. What is the internal working of HashMap?

Interview-ready explanation: HashMap stores entries in an array of bins. It spreads the key's hash, computes a bin index, then uses hash and equals() to find/update the exact key; collisions share a bin and can be represented as linked nodes or, under Java 8+ thresholds, a red-black tree.

Why / where it is used: Understanding this explains average O(1) lookup, why good hashCode() implementations matter, why mutable keys are dangerous and why load factor/resizing influence performance.

HashMap stores entries in an array of buckets (bins). A key's hash is spread, an index is calculated, and the entry is placed in that bin. Collisions are stored in the same bin as a linked structure and may become a red-black tree in modern Java when collision thresholds are met.

- put(): compute hash -> bucket index -> compare keys using hash + equals() -> insert/update.
- get(): compute same hash/index -> scan/tree-search the bin -> equals() confirms the key.
- Average get/put is O(1); heavy collisions can degrade performance.
- Java 8+ can treeify heavily-collided bins (commonly threshold 8, with capacity conditions).

[VERY IMPORTANT] Q30. HashMap vs synchronizedMap vs ConcurrentHashMap

Interview-ready explanation: HashMap provides no thread-safety. Collections.synchronizedMap() wraps a map with one monitor, so most operations are serialized; ConcurrentHashMap is designed for high concurrency with lock-free/low-lock reads, CAS and fine-grained bin synchronization for updates.

Why / where it is used: Use HashMap for single-threaded or externally synchronized state, synchronizedMap for simple low-concurrency legacy cases, and ConcurrentHashMap for shared maps in servers, caches and registries.

HashMap	Collections.synchronizedMap(...)	ConcurrentHashMap
Not thread-safe	Thread-safe wrapper with coarse synchronization	Designed for high concurrency
Allows one null key + null values	Same null behavior as wrapped map	No null keys or values
Fail-fast iterators	Iteration requires external synchronization for a consistent traversal	Weakly consistent iterators
Best for single-threaded/external synchronization	Simple synchronized access	Best general concurrent map

Q31. How can you put all elements into the same HashMap bucket?

Interview-ready explanation: If every custom key returns the same hash code, HashMap sends those keys to the same logical bin (assuming they map to the same index). equals() must still distinguish them or later puts will replace the earlier key.

Why / where it is used: This is useful only as a learning/test scenario for collision handling. Real hashCode() implementations should distribute keys well to avoid collision-heavy performance.

Create keys whose `hashCode()` always returns the same value while `equals()` still distinguishes different keys. This intentionally creates collisions.

```
final class BadKey {
    private final int id;
    BadKey(int id) { this.id = id; }

    @Override public int hashCode() { return 1; }
    @Override public boolean equals(Object o) {
        return o instanceof BadKey other && id == other.id;
    }
}
```

NOTE: This is a demonstration only. In Java 8+, a heavily-collided bin may treeify rather than remain a simple linked list.

Q32. `hashCode()` and `equals()` contract

Interview-ready explanation: `equals()` defines logical equality and `hashCode()` supports hash-based lookup. The key contract is: equal objects must produce equal hash codes; unequal objects may still collide. `equals()` must also be reflexive, symmetric, transitive and consistent.

Why / where it is used: If an object is used as a `HashMap` key, do not mutate fields that participate in `equals/hashCode` while the key is stored, because lookup may later search the wrong bin.

- `equals()` is reflexive, symmetric, transitive, consistent, and false for null.
- If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must be true.
- Unequal objects may have the same hash code.
- Fields used by `equals()` should normally also be used consistently in `hashCode()`.
- Avoid mutating key fields that participate in `equals/hashCode` while the object is stored as a `HashMap` key.

Q33. How many default buckets and what load factor does `HashMap` have?

Interview-ready explanation: The commonly cited `HashMap` defaults are an initial capacity of 16 and load factor 0.75. The internal table is allocated lazily in modern Java, and resizing normally occurs when size exceeds capacity $\times$ loadFactor.

Why / where it is used: These defaults balance memory use and collision probability. Pre-sizing a map can help when you already know it will hold many entries and want to avoid repeated resizing.

The commonly cited default initial capacity is 16 and the default load factor is 0.75. The resize threshold is capacity $\times$ loadFactor, so for capacity 16 the threshold is 12 entries (subject to implementation details and lazy table allocation).

Q34. Why do we need multiple buckets in `HashMap`?

Interview-ready explanation: Multiple bins let `HashMap` use the hash to narrow a lookup to a small subset of entries instead of scanning the entire collection. With a good distribution, most bins stay small, producing near-constant average get/put time.

Why / where it is used: A poor hash distribution collapses many entries into a few bins, increasing comparisons and making the map slower.

Buckets distribute entries based on hashes so lookup can jump to a small subset of entries instead of scanning the entire map. Good distribution gives average $O(1)$ get/put performance.

Q35. How does HashSet work internally?

Interview-ready explanation: HashSet is implemented on top of HashMap: each set element becomes a map key and all keys share the same dummy value. Therefore the uniqueness rules of HashSet are exactly the key-equality rules of HashMap.

Why / where it is used: This is why equals() and hashCode() are crucial for custom objects stored in a set and why HashSet has similar average performance characteristics to HashMap keys.

HashSet is backed by a HashMap. The Set element is stored as the HashMap key and all entries share a dummy PRESENT value.

```
// Conceptually inside HashSet
private transient HashMap<E, Object> map;
private static final Object PRESENT = new Object();

public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}
```

INTERVIEW TIP: Uniqueness in HashSet is ultimately determined using hashCode() and equals().

Q36. Fail-Fast vs “Fail-Safe” iterators

Interview-ready explanation: Fail-fast iterators detect unexpected structural modification and may throw ConcurrentModificationException; they are common in ArrayList and HashMap. Concurrent collections instead provide iterator semantics designed to tolerate concurrent change, such as snapshot iteration in CopyOnWriteArrayList or weak consistency in ConcurrentHashMap.

Why / where it is used: 'Fail-safe' is common interview terminology, not an official Iterator API category. Also, fail-fast detection is best-effort and must not be used as a thread-safety mechanism.

Fail-fast	Concurrent / commonly called fail-safe
Typical ArrayList/HashMap iterators	Concurrent collections tolerate concurrent changes
May throw ConcurrentModificationException on unexpected structural modification	Usually do not throw CME just because another thread modifies the collection
Best-effort detection, not a correctness guarantee	May be snapshot-based or weakly consistent
Iterator.remove() is the supported iterator-side removal	CopyOnWriteArrayList: snapshot; ConcurrentHashMap: weakly consistent

NOTE: “Fail-safe” is common interview terminology, not an official Java Collections API classification.

Q37. What happens if I add a new element to a Map while iterating it?

Interview-ready explanation: Adding a new key to a normal HashMap while traversing one of its fail-fast iterators is a structural modification and typically causes ConcurrentModificationException on a later iterator operation. Updating an existing value is different because it may not change the map's structure.

Why / where it is used: Use Iterator.remove() for supported iterator removal, collect changes and apply them later, or use a concurrent collection such as ConcurrentHashMap when real concurrent mutation is required.

With a normal HashMap iterator, adding a new key through map.put() is a structural modification and will typically cause ConcurrentModificationException on a later iterator operation. Updating only the value of an existing key is not necessarily structural. Iterator.remove() is the supported removal mechanism. ConcurrentHashMap allows concurrent modifications with a weakly consistent iterator.

```
for (Map.Entry<Integer,String> e : map.entrySet()) {  
    map.put(99, "new"); // HashMap: typically ConcurrentModificationException  
}
```

[VERY IMPORTANT] Q38. ConcurrentHashMap bucket-level locking

Interview-ready explanation: Java 8+ ConcurrentHashMap does not use the old Java 7 Segment design. Reads are highly concurrent; insertion into an empty bin can use CAS, and contended updates synchronize only on the relevant bin/tree structure rather than locking the whole map.

Why / where it is used: This fine-grained design is why ConcurrentHashMap is suitable for shared caches, counters and registries under heavy multi-threaded access.

In Java 8+, ConcurrentHashMap no longer uses the Java 7 Segment architecture. It combines CAS with fine-grained synchronization on individual bins/nodes when necessary, so the entire map is not locked for normal updates.

- Reads are highly concurrent and usually do not lock.
- Empty-bin insertion can use CAS.
- Contended updates may synchronize on the bin head.
- Tree bins are used for heavy collisions under implementation thresholds.
- Java 7 used segment-level locking.

Q39. Array vs ArrayList

Interview-ready explanation: An array is a fixed-length language/runtime construct that can store primitives or references. ArrayList is a resizable List backed by an object array and provides collection operations such as add/remove/contains/iteration utilities.

Why / where it is used: Choose arrays for fixed-size, low-overhead structures, primitive buffers and multidimensional data; choose ArrayList for general application collections that grow/shrink and benefit from the Collections API.

Array	ArrayList
Fixed length after creation	Dynamic logical size
Can store primitives directly	Stores objects; primitives use wrappers such as Integer
Lower overhead and direct indexed storage	Convenient add/remove/search APIs
Covariant and runtime type-checked	Generic and compile-time type-safe
Useful for fixed-size/performance-sensitive data	General-purpose resizable list

Q40. If an array has fixed size, why use an array instead of ArrayList?

Interview-ready explanation: Fixed size is sometimes exactly what you want. Arrays have low overhead, can store primitives without boxing, offer predictable contiguous indexed storage and work naturally with APIs/algorithms that expect arrays.

Why / where it is used: ArrayList adds flexibility and convenience, but that flexibility costs extra object/boxing/resizing overhead and is unnecessary when the size is known and stable.

- Lower memory overhead.
- Can store primitives without boxing.
- Good cache locality and predictable fixed size.
- Fast indexed access with minimal abstraction.
- Required by many APIs/algorithms and useful for matrices/buffers.
- If the size truly is fixed, dynamic resizing features add no value.

[VERY IMPORTANT] Q41. Comparable vs Comparator

Interview-ready explanation: Comparable puts a type's natural ordering inside the class through compareTo(). Comparator is an external strategy object/function that can define many different orderings without changing the domain class.

Why / where it is used: Use Comparable when there is one obvious natural order, such as dates or IDs. Use Comparator for UI/business sorts such as Employee by salary, name, department or multiple chained fields.

Comparable	Comparator
Class implements Comparable<T>	Separate Comparator<T> object/function
compareTo(T)	compare(T,T)
Defines natural/default order	Defines custom/multiple orders
Requires modifying the class	Can sort third-party classes
Example: String natural order	Example: compare employees by salary, then name

```
employees.sort(Comparator.comparing(Employee::getSalary)
    .thenComparing(Employee::getName));
```

Q42. Generics and type erasure

Interview-ready explanation: Generics provide compile-time type safety and reusable parameterized APIs, but most generic type parameters are erased from runtime class metadata. The compiler replaces type variables with their bounds and inserts casts/bridge methods where necessary.

Why / where it is used: Type erasure explains why List<String> and List<Integer> share the same runtime class and why operations such as new T(), new T[] and instanceof List<String> are not directly allowed.

Generics provide compile-time type safety and reusable parameterized types. Java implements most generics through type erasure: generic type parameters are removed/replaced by their bounds at runtime, with casts/bridge methods inserted as needed.

- List<String> and List<Integer> are the same raw List class at runtime.
- You cannot directly create new T() or new T[] because the concrete type is erased.
- Overloads that differ only by erased generic type can clash.

Q43. PECS: ? extends vs ? super

Interview-ready explanation: PECS means Producer Extends, Consumer Super. A ? extends T reference is ideal when you only need to read values as T, while ? super T is ideal when you need to add T values safely.

Why / where it is used: This rule is central to writing flexible generic APIs such as copy methods. It lets callers pass lists of subtypes or supertypes without giving up compile-time safety.

PECS means Producer Extends, Consumer Super.

- ? extends T: use when a structure produces T values for you to read.
- ? super T: use when a structure consumes T values you want to add.

```
List<? extends Number> src; // Read Numbers safely; do not add specific Number values  
List<? super Integer> dst; // Can add Integers; reads are Object
```

5. Functional Interfaces & Streams

Q44. How many abstract methods can a functional interface have?

Interview-ready explanation: A functional interface has exactly one abstract method, also called a SAM. It may still contain default methods, static methods and Object-compatible declarations without losing functional-interface status.

Why / where it is used: Functional interfaces are the target types for lambda expressions and method references. Examples include Runnable, Comparator, Predicate and Function.

Exactly one abstract method (SAM: Single Abstract Method). It may still have any number of default/static methods. Methods matching public Object methods do not count toward the single abstract method rule.

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

Q45. Functional Interface vs Lambda expression

Interview-ready explanation: A functional interface is a type/contract; a lambda is one concise way to provide an implementation of that contract. The lambda gets its parameter/return types from the target functional interface.

Why / where it is used: This distinction matters because the same lambda syntax can target different compatible interfaces, and APIs such as Stream accept functional-interface parameters rather than 'lambdas' as a runtime type.

Functional Interface	Lambda
A type/contract with one abstract method	A concise implementation of that SAM
Can declare default/static methods	Has no standalone target type; it needs a compatible functional interface
Example: Predicate<T>	Example: x -> x > 0

Q46. Predicate, Supplier, Consumer, Function

Interview-ready explanation: Predicate<T> tests a condition and returns boolean; Supplier<T> produces a value with no input; Consumer<T> accepts a value and returns nothing; Function<T,R> transforms T into R.

Why / where it is used: These four interfaces appear throughout Streams, Optional, CompletableFuture and callback-style APIs. Learn their shapes because interviewers often ask you to choose the right one for a scenario.

Interface	Shape	Use
Predicate<T>	T -> boolean	Filtering/testing
Supplier<T>	() -> T	Produce a value
Consumer<T>	T -> void	Consume/perform side effect
Function<T,R>	T -> R	Transform one value into another

```
Predicate<Integer> even = n -> n % 2 == 0;
Supplier<String> token = () -> "abc";
Consumer<String> print = System.out::println;
Function<String,Integer> length = String::length;
```

Q47. map() vs flatMap() in Stream API

Interview-ready explanation: map() transforms each stream element into exactly one result element. flatMap() transforms each element into another stream/collection-like sequence and then flattens those nested sequences into one stream.

Why / where it is used: Use map for one-to-one transformations such as Employee -> name. Use flatMap for nested collections, Optional/Stream composition or operations where one input may produce zero/many outputs.

map()	flatMap()
One input element -> one transformed output element	One input -> stream/collection of outputs, then flattens
Can produce Stream<List<T>>	Can flatten to Stream<T>
Use for simple transformation	Use for nested structures

```
List<List<Integer>> data = List.of(List.of(1,2), List.of(3,4));
List<Integer> flat = data.stream()
    .flatMap(List::stream)
    .toList(); // [1,2,3,4]
```

Q48. Can we call stream() directly on an int[] array?

Interview-ready explanation: Primitive arrays do not have an instance stream() method. Use Arrays.stream(int[]) or IntStream.of(...) to obtain an IntStream; object arrays can use Arrays.stream(T[]) or Stream.of(...).

Why / where it is used: Primitive streams avoid boxing overhead and provide numeric operations such as sum(), average() and max().

No instance stream() method exists on arrays. Use Arrays.stream(arr) or IntStream.of(arr).

```
int[] arr = {1,2,3};
IntStream s1 = Arrays.stream(arr);
IntStream s2 = IntStream.of(arr);
```

6. Exceptions & Resource Management

Q49. Exception hierarchy

Interview-ready explanation: Throwable is the root of Java's throwable hierarchy. Error represents serious JVM/environment problems, while Exception represents conditions applications may handle; RuntimeException and its subclasses are unchecked.

Why / where it is used: Understanding the hierarchy tells you what the compiler forces you to catch/declare and helps you decide whether a failure represents a recoverable business condition, programming bug or system-level problem.

- Error: serious JVM/system conditions applications generally do not recover from.
- Checked exceptions: compiler enforces catch/declare.
- RuntimeException and subclasses: unchecked programming/runtime failures.

```
Object
├─ Throwable
│   ├── Error
│   │   └─ OutOfMemoryError, StackOverflowError, ...
│   └─ Exception
│       ├── RuntimeException // unchecked
│       └─ Other Exceptions // checked
```

Q50. Checked vs unchecked exceptions

Interview-ready explanation: Checked exceptions are enforced by the compiler: the caller must catch them or declare them. Unchecked exceptions are RuntimeException subclasses and are not subject to that rule; they commonly represent programming errors or invalid runtime state.

Why / where it is used: Use checked exceptions selectively for recoverable conditions a caller is expected to handle. Overusing checked exceptions can make APIs noisy, while using unchecked exceptions for everything can hide meaningful recovery paths.

Checked	Unchecked
Subclass of Exception excluding RuntimeException	RuntimeException subclasses (and Errors are also unchecked by compiler rules)
Compiler requires catch or throws declaration	No compile-time catch/declare requirement
Example: IOException, SQLException	Example: NullPointerException, IllegalArgumentException

Q51. throw vs throws

Interview-ready explanation: throw is an executable statement that actually throws one exception object. throws appears in a method/constructor signature to declare that the operation may propagate one or more exception types.

Why / where it is used: A method may both declare throws and explicitly throw. The distinction is foundational to exception propagation and API contracts.

throw	throws
Actually throws a Throwable object	Declares that a method may propagate exceptions

throw	throws
Used inside method/block	Used in method/constructor signature
throw new IllegalArgumentException();	void read() throws IOException

Q52. Exception handling rules with method overriding

Interview-ready explanation: An overriding method must preserve the checked-exception contract of the parent. It can declare the same checked exception, a subclass/narrower checked exception, or none, but not a broader checked exception; unchecked exceptions are not restricted in the same way.

Why / where it is used: This protects substitutability: code written against the parent type must not suddenly be forced to handle a new broader checked exception from a child implementation.

- Overriding method cannot declare a broader checked exception than the parent method.
- It may declare the same checked exception, a subclass, or no checked exception.
- Unchecked exceptions may be added without this restriction.

```
class Parent { void read() throws IOException { } }
class Child extends Parent {
    @Override void read() throws FileNotFoundException { } // Allowed: narrower
}
```

Q53. What is try-with-resources?

Interview-ready explanation: try-with-resources manages AutoCloseable resources automatically. Resources are closed in reverse declaration order even when the body throws, and any close failures can be attached as suppressed exceptions.

Why / where it is used: Use it for files, streams, sockets, JDBC connections/statements/result sets and other closeable resources. It is safer and clearer than hand-written finally cleanup.

try-with-resources automatically closes resources that implement AutoCloseable. Resources close in reverse declaration order, even when exceptions occur.

- Reduces boilerplate finally blocks.
- If both body and close() throw, the close exception is available as a suppressed exception.
- Since Java 9, an effectively-final existing resource can be referenced directly.

```
try (BufferedReader br = Files.newBufferedReader(path)) {
    return br.readLine();
}
```

Q54. AutoCloseable vs Closeable

Interview-ready explanation: AutoCloseable is the general resource contract with close() that may throw Exception. Closeable is an older I/O-specific subinterface whose close() throws IOException and is expected to be idempotent by convention.

Why / where it is used: Most modern resource types can participate in try-with-resources through AutoCloseable; I/O classes commonly implement Closeable.

AutoCloseable	Closeable
java.lang.AutoCloseable	java.io.Closeable extends AutoCloseable

AutoCloseable	Closeable
close() may throw Exception	close() throws IOException
General-purpose resource contract	Primarily I/O resources
No strict idempotence requirement	Closeable contract expects repeated close to have no effect

7. Serialization, Copying & Object Lifecycle

Q55. Serialization vs deserialization

Interview-ready explanation: Serialization converts an object's serializable state into a byte representation; deserialization reconstructs an object from that representation. Java's native Serializable mechanism uses metadata such as serialVersionUID and skips static and transient state by default.

Why / where it is used: Native Java serialization is still interview-relevant and appears in legacy systems, but modern services often prefer JSON, protobuf or other explicit formats because they are safer and more interoperable.

- serialVersionUID is used for version compatibility.
- static fields are class state and are not serialized as object state.
- transient fields are skipped by default.

Serialization	Deserialization
Object -> byte stream	Byte stream -> object
ObjectOutputStream	ObjectInputStream
Class typically implements Serializable	Reconstructs serialized object graph

Q56. Serialization with inheritance

Interview-ready explanation: Serializable state follows the inheritance hierarchy. If a parent is Serializable, its serializable fields participate; if the child is Serializable but the parent is not, the non-serializable parent portion is initialized through the parent's accessible no-arg constructor rather than restored from the stream.

Why / where it is used: This explains surprising constructor behavior during deserialization and why legacy serializable class hierarchies need careful versioning.

- If a parent implements Serializable, serializable state of child objects is included automatically.
- If only the child implements Serializable and the parent does not, the non-serializable parent state is not restored from the stream; the parent no-arg constructor runs during deserialization.
- Therefore a non-serializable parent needs an accessible no-arg constructor for normal Java serialization.
- transient/static fields are not serialized as normal instance state.

Q57. Shallow copy vs deep copy

Interview-ready explanation: A shallow copy creates a new outer object but reuses references to nested mutable objects. A deep copy recursively creates independent nested objects so mutations in the copy do not affect the original.

Why / where it is used: Shallow copying is cheaper and sufficient when nested state is immutable/shared intentionally. Deep copying is needed when the copy must be fully independent, but it costs more time and memory and requires a clear copy strategy.

Shallow copy	Deep copy
Copies top-level object, nested references are shared	Copies the object and recursively copies mutable nested state
Changing shared child may affect both copies	Copies are independent
Faster/cheaper	More work/memory
Object.clone() is shallow by default for references	Usually implemented manually, copy constructors, mapping, or serialization-like techniques

8. Multithreading & Concurrency

Q58. Difference between wait(), sleep() and yield()

Interview-ready explanation: sleep() pauses the current thread for a duration without releasing locks. wait() is monitor coordination: the thread must own the monitor, releases it while waiting and later reacquires it. yield() is only a scheduling hint that the current thread is willing to let another runnable thread execute.

Why / where it is used: Use wait/notify only for low-level monitor coordination; most production code should prefer higher-level concurrency utilities. sleep is for timing/backoff, while yield has few reliable business use cases.

wait()	sleep()	yield()
Object method; must own monitor	Thread static method	Thread static scheduler hint
Releases the monitor while waiting	Does not release held monitors	Does not release held monitors
Waits for notify/notifyAll, interrupt or timeout	Waits for time or interrupt	Hints scheduler to let another runnable thread run
Used for coordination on a condition	Used for delay/backoff	No scheduling guarantee

Q59. Why are wait(), notify(), and notifyAll() defined in Object instead of Thread?

Interview-ready explanation: wait(), notify() and notifyAll() operate on an object's monitor and its wait set, not on a Thread object. Since any Java object can be used as a synchronization monitor, these methods logically belong to Object.

Why / where it is used: The calling thread must own that same object's monitor, typically inside synchronized(obj); otherwise IllegalMonitorStateException is thrown.

Because the coordination is tied to an object monitor/lock, not to the Thread object. Any Java object can act as the monitor that threads synchronize on, so the monitor wait-set operations belong to Object.

INTERVIEW TIP: wait()/notify()/notifyAll() must be called while holding that same object's monitor, normally inside synchronized(obj).

Q60. Difference between sleep(), wait() and join()

Interview-ready explanation: sleep() pauses the current thread for time and keeps any held monitor. wait() releases a monitor and waits for notification/timeout. join() makes one thread wait until another thread terminates (or the join timeout expires).

Why / where it is used: These methods solve different problems: timing, monitor coordination and lifecycle coordination. Confusing whether a lock is released is a common interview trap.

sleep()	wait()	join()
Pause current thread for time	Wait on an object condition	Wait for another thread to finish
Thread.sleep	Object.wait	Thread.join
Does not release monitor	Releases the monitor being waited on	Caller waits until target thread terminates or timeout/interruption

sleep()	wait()	join()
No synchronized block required	Must own object monitor	No monitor ownership requirement

Q61. What is ThreadLocal?

Interview-ready explanation: ThreadLocal associates a separate value with each thread even though all threads reference the same ThreadLocal variable. Reads/writes affect only the current thread's slot.

Why / where it is used: It is useful for per-request correlation IDs, legacy transaction/security context and non-thread-safe per-thread helpers. In thread pools, always remove values when finished to prevent stale data retention/leaks.

ThreadLocal gives each thread its own independent value associated with the same ThreadLocal variable. It is useful for per-thread context when passing a parameter everywhere would be awkward.

```
private static final ThreadLocal<String> REQUEST_ID = new ThreadLocal<>();
try {
    REQUEST_ID.set("req-123");
    // use REQUEST_ID.get()
} finally {
    REQUEST_ID.remove(); // Important for pooled threads
}
```

NOTE: With thread pools, forgetting remove() can retain stale data and contribute to memory leaks.

[VERY IMPORTANT] Q62. volatile vs Atomic classes

Interview-ready explanation: volatile guarantees visibility and ordering for reads/writes of one variable, but it does not make compound read-modify-write operations atomic. Atomic classes such as AtomicInteger provide atomic CAS-based operations like incrementAndGet() without a synchronized block.

Why / where it is used: Use volatile for flags or independently assigned state; use atomics for counters/state transitions. For multi-variable invariants or larger critical sections, use locks/synchronization instead.

volatile	AtomicInteger / AtomicReference etc.
Visibility + ordering guarantee for a variable	Atomic read-modify-write operations
Read/write of the variable is visible across threads	Methods such as incrementAndGet(), compareAndSet()
count++ is NOT made atomic	count.incrementAndGet() is atomic
Good for flags/state publication	Good for lock-free counters/state transitions

Q63. CountdownLatch vs CyclicBarrier

Interview-ready explanation: CountdownLatch is a one-shot gate: threads wait until a counter reaches zero. CyclicBarrier is reusable and makes a fixed number of threads meet at a barrier point before any of them continue.

Why / where it is used: Use a latch for 'wait until N startup/tasks complete'. Use a barrier for phased parallel algorithms where peers repeatedly synchronize at the end of each phase.

CountDownLatch	CyclicBarrier
One or more threads wait until count reaches zero	A fixed number of threads wait for each other at a barrier
countDown() + await()	await()
One-shot; cannot reset	Reusable after barrier trips
Good for “wait until N tasks finish”	Good for phased parallel work

[VERY IMPORTANT] Q64. Future vs CompletableFuture

Interview-ready explanation: Future represents a result that may arrive later, but its API is mainly blocking/polling through get()/isDone(). CompletableFuture is both a Future and a composable completion stage, supporting callbacks, transformations, combination, error handling and asynchronous pipelines.

Why / where it is used: Use CompletableFuture when several independent I/O/computation steps must be composed without manually blocking a thread after every step.

Future	CompletableFuture
Represents one asynchronous result	Future plus fluent asynchronous composition
get() commonly blocks	thenApply/thenCompose/thenCombine/allOf enable pipelines
Limited chaining/error handling	Rich exception handling and callbacks
Usually obtained from ExecutorService.submit()	Can be created with runAsync/supplyAsync or completed manually

Q65. CompletableFuture: runAsync() vs supplyAsync()

Interview-ready explanation: runAsync() executes a Runnable-style action and therefore returns CompletableFuture<Void>. supplyAsync() executes a Supplier-style task and returns CompletableFuture<T> containing the produced value.

Why / where it is used: Use runAsync for fire-and-continue side effects and supplyAsync when later stages need a result to transform or combine.

runAsync()	supplyAsync()
Runs Runnable	Runs Supplier<T>
No result value	Produces a result
CompletableFuture<Void>	CompletableFuture<T>

```
CompletableFuture<Void> a = CompletableFuture.runAsync() -> doWork();
CompletableFuture<String> b = CompletableFuture.supplyAsync() -> loadData();

b.thenApply(String::toUpperCase)
.exceptionally(ex -> "fallback");
```

Q66. Object-level lock vs class-level lock

Interview-ready explanation: An object-level synchronized instance method/block locks a particular object's monitor, so different instances can proceed independently. A static synchronized method or synchronized(ClassName.class) locks the Class object and therefore coordinates all instances of that class.

Why / where it is used: Choose the smallest lock scope that protects the shared state. Using a class-level lock for instance-local data unnecessarily reduces concurrency.

Object-level lock	Class-level lock
Locks one instance monitor	Locks the Class object monitor
synchronized instance method / synchronized(this)	static synchronized / synchronized(MyClass.class)
Different instances can execute concurrently	All instances share the same class lock for that critical section

Q67. What is ReentrantLock?

Interview-ready explanation: ReentrantLock is an explicit mutual-exclusion lock. It is reentrant, meaning the same thread can acquire it repeatedly, and it offers capabilities synchronized does not directly expose, such as tryLock(), interruptible acquisition, optional fairness and multiple Condition objects.

Why / where it is used: Use it when you need those advanced capabilities or explicit lock control. Always unlock in finally to avoid permanently blocking other threads.

ReentrantLock is an explicit lock from java.util.concurrent.locks. “Reentrant” means the same thread can acquire the same lock multiple times.

- Supports tryLock(), lockInterruptibly(), optional fairness and multiple Condition objects.
- Always unlock in finally.

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

Q68. How do you achieve thread safety in a multithreaded environment?

Interview-ready explanation: Thread safety starts by minimizing shared mutable state. When sharing is necessary, use immutability/thread confinement, safe publication, synchronized/Lock, atomic variables and concurrent collections according to the size of the invariant you must protect.

Why / where it is used: In real systems also prefer high-level executors, BlockingQueue, futures and structured APIs over hand-written wait/notify because they reduce race-condition complexity.

- Prefer immutability and avoid shared mutable state.
- Use thread confinement / local variables where possible.
- Use synchronized or explicit Lock for critical sections.
- Use atomics for simple atomic state transitions/counters.

- Use concurrent collections such as ConcurrentHashMap and BlockingQueue.
- Use safe publication (final fields, volatile, synchronization, concurrent containers).
- Use higher-level executors/futures instead of manual thread coordination when possible.

Q69. Explain ExecutorService and types of executors

Interview-ready explanation: ExecutorService separates task submission from thread lifecycle management. You submit Runnable/Callable work, the executor schedules it on managed workers, and you control shutdown as a service rather than creating ad-hoc Thread objects everywhere.

Why / where it is used: Common choices include fixed pools, cached pools, scheduled executors, single-thread executors and, in modern Java, virtual-thread-per-task executors. Choose based on blocking behavior, throughput and resource limits.

ExecutorService separates task submission from thread management. You submit Runnable/Callable tasks and the executor manages worker threads and lifecycle.

- Call shutdown() for orderly shutdown; awaitTermination() if needed.

Factory / Executor	Typical use
newFixedThreadPool(n)	Bounded number of reusable workers
newCachedThreadPool()	Creates/reuses threads as needed; can grow aggressively
newSingleThreadExecutor()	Sequential task execution
newScheduledThreadPool(n)	Delayed/periodic tasks
newWorkStealingPool()	ForkJoinPool-based work stealing
newVirtualThreadPerTaskExecutor() (Java 21+)	One virtual thread per submitted task; useful for high-concurrency blocking I/O

Q70. newCachedThreadPool() vs newFixedThreadPool()

Interview-ready explanation: newFixedThreadPool(n) keeps at most n worker threads and queues additional tasks. newCachedThreadPool() can create many threads as demand rises and reuses idle ones, which is flexible but dangerous under unbounded load.

Why / where it is used: Use fixed/bounded executors when you need resource control. Cached pools can fit short-lived bursty work but can overwhelm memory/downstream systems if task arrival is not constrained.

Cached	Fixed
Potentially creates many threads when load grows	At most N worker threads
Uses idle thread reuse	Tasks wait in queue when all workers busy
Good for many short-lived tasks only when growth is controlled	Predictable concurrency/resource limit
Risk: unbounded thread growth under sustained load	Risk: queue growth if producers outpace workers

Q71. Explain deadlock

Interview-ready explanation: A deadlock is a permanent cycle of threads waiting for locks/resources held by each other. The classic necessary conditions are mutual exclusion, hold-and-wait, no preemption and circular wait.

Why / where it is used: Prevent it with consistent lock ordering, fewer nested locks, timed tryLock() and small critical sections. Diagnose production deadlocks with thread dumps, jcmd/jstack, JFR or profilers.

A deadlock occurs when threads are permanently waiting for locks/resources held by each other.

- Classic conditions: mutual exclusion, hold-and-wait, no preemption, circular wait.
- Prevention: consistent lock ordering, reduce nested locking, use tryLock/timeouts, minimize critical sections.
- Diagnosis: thread dumps (jstack/jcmd), JFR, profilers.

```
// Risk: opposite lock order  
Thread 1: lock(A) -> lock(B)  
Thread 2: lock(B) -> lock(A)
```

Q72. ForkJoinPool and work stealing

Interview-ready explanation: ForkJoinPool targets divide-and-conquer CPU work. Tasks split into smaller tasks; each worker keeps a deque, and idle workers steal tasks from others to balance load dynamically.

Why / where it is used: Use RecursiveTask/RecursiveAction or parallel-stream style CPU workloads. It is less suitable for long blocking I/O unless blocking is carefully managed.

ForkJoinPool is designed for recursive/parallel tasks. Work is split into subtasks; each worker has a deque. An idle worker can steal tasks from another worker's deque, improving CPU utilization.

- RecursiveTask<V>: returns a result.
- RecursiveAction: no result.
- Best suited to CPU-bound divide-and-conquer work; avoid blocking tasks unless designed appropriately.

Q73. Write a program where one thread prints odd numbers and another prints even numbers

Interview-ready explanation: The core idea is to protect one shared counter with a monitor and have each thread wait until the counter parity belongs to it. Always check the wait condition in a while loop, update state while holding the lock and notify waiting peers after progress.

Why / where it is used: This question tests monitor ownership, spurious wakeups, visibility and coordination rather than just printing numbers.

```

class OddEvenPrinter {
    private int n = 1;
    private final int max;
    OddEvenPrinter(int max) { this.max = max; }

    synchronized void printOdd() throws InterruptedException {
        while (n <= max) {
            while (n <= max && n % 2 == 0) wait();
            if (n <= max) System.out.println("Odd : " + n++);
            notifyAll();
        }
    }

    synchronized void printEven() throws InterruptedException {
        while (n <= max) {
            while (n <= max && n % 2 != 0) wait();
            if (n <= max) System.out.println("Even : " + n++);
            notifyAll();
        }
    }

    public static void main(String[] args) {
        OddEvenPrinter p = new OddEvenPrinter(10);
        new Thread() -> { try { p.printOdd(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); } }).start();
        new Thread() -> { try { p.printEven(); } catch (InterruptedException e)
        { Thread.currentThread().interrupt(); } }).start();
    }
}

```

Q74. Implement a bounded blocking queue using wait() and notifyAll()

Interview-ready explanation: A bounded blocking queue combines a finite buffer with two conditions: producers must wait while the queue is full and consumers must wait while it is empty. Both conditions must be checked in while loops under the same monitor, and successful put/take operations notify waiting peers.

Why / where it is used: In production, prefer ArrayBlockingQueue/LinkedBlockingQueue; implementing it manually is an interview exercise for wait/notify and producer-consumer design.

```

class SimpleBlockingQueue<T> {
    private final Queue<T> q = new ArrayDeque<>();
    private final int capacity;
    SimpleBlockingQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (q.size() == capacity) wait();
        q.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        T item = q.remove();
        notifyAll();
        return item;
    }
}

```

INTERVIEW TIP: Use while, not if, around wait() because of spurious wakeups and because another thread may consume the condition before you reacquire the lock. In production, prefer java.util.concurrent.BlockingQueue.

Q75. Write a Java program to print a sequence up to N using 3 threads in order

Interview-ready explanation: To print 1..N with three threads in order, keep one shared current number and let each thread run only when (number-1) % 3 matches its thread index. The shared condition must be protected by synchronization and waiting threads awakened after each increment.

Why / where it is used: This generalizes the odd/even problem and tests condition-based coordination among more than two workers.

```
class ThreeThreadSequence {
    private int n = 1;
    private final int max;
    ThreeThreadSequence(int max) { this.max = max; }

    synchronized void print(int turn) throws InterruptedException {
        while (n <= max) {
            while (n <= max && (n - 1) % 3 != turn) wait();
            if (n <= max) {
                System.out.println("THREAD-" + (turn + 1) + " : " + n++);
                notifyAll();
            }
        }
    }
}

public static void main(String[] args) {
    ThreeThreadSequence p = new ThreeThreadSequence(10);
    for (int i = 0; i < 3; i++) {
        int turn = i;
        new Thread() -> {
            try { p.print(turn); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }.start();
    }
}
```

9. JVM Memory Model, OOM & Garbage Collection

Q76. What is the Java Memory Model (JMM)?

Interview-ready explanation: The Java Memory Model defines how threads interact through memory: when writes by one thread become visible to another and what reorderings are legal. The key concept is the happens-before relationship created by synchronization actions such as monitor unlock/lock, volatile write/read, thread start/join and final-field construction rules.

Why / where it is used: JMM knowledge explains visibility bugs, safe publication, double-checked locking and why 'it works on my machine' is not proof that unsynchronized shared code is correct.

The JMM defines the rules for how threads interact through memory: visibility, ordering, and atomicity of reads/writes. It explains when one thread is guaranteed to see another thread's actions.

- happens-before relationships provide visibility/order guarantees.
- synchronized establishes lock/unlock happens-before relationships.
- volatile write happens-before a later volatile read of the same variable.
- Thread start/join and final-field rules also provide guarantees.

NOTE: JMM is not the same thing as JVM runtime memory areas such as heap, stack, metaspace and native memory.

Q77. What can cause OutOfMemoryError and how do you fix it?

Interview-ready explanation: OutOfMemoryError means the JVM cannot satisfy a memory allocation in a particular area or native resource. Causes include a genuinely undersized heap, retained-object leaks, excessive direct/native buffers, metaspace growth, thread creation and huge allocations.

Why / where it is used: Fix the cause, not just the heap size: capture heap dumps, inspect dominator/retained-size paths with tools such as VisualVM, JProfiler, Eclipse MAT or JFR, review GC logs and verify object/thread/classloader lifecycles.

Diagnosis/fix: capture the exact OOM message, heap dump and GC/JFR data; identify dominant retainers; fix leaks/unbounded caches or right-size memory only after understanding the cause.

- Java heap exhausted: true workload pressure or retained-object leak.
- Metaspace exhaustion: excessive class loading/classloader leaks.
- Direct/native memory exhaustion: direct buffers/native allocations.
- Unable to create native thread: too many threads or OS/native-memory limits.
- GC overhead limit: JVM spends excessive time collecting with little reclaimed memory.

Useful options/tools:

-XX:+HeapDumpOnOutOfMemoryError

jcmd / jmap / jstack / JFR

VisualVM (the question bank says "VirtualVM")

JProfiler

Q78. What is a memory leak in Java and how do you prevent it?

Interview-ready explanation: A Java memory leak occurs when objects are no longer useful to the application but are still strongly reachable, so GC is not allowed to reclaim them. Common causes are

unbounded caches/maps, forgotten listeners, static collections, ThreadLocal values, classloader retention and resources holding references.

Why / where it is used: Use bounded/evicting caches, remove listeners/ThreadLocals, close resources, avoid unnecessary static state and validate with heap dumps/profilers when retained memory grows unexpectedly.

A Java memory leak means objects are no longer useful to the application but are still strongly reachable, so GC cannot reclaim them.

- Bound caches/collections; avoid ever-growing static maps.
- Close resources with try-with-resources.
- Remove listeners/callback registrations when no longer needed.
- Use ThreadLocal.remove() in pooled-thread scenarios.
- Avoid accidental classloader retention.
- Profile heap dominator/retained-size paths with JProfiler, VisualVM, JFR or heap analyzers.

Q79. Explain mark-sweep garbage collection algorithm

Interview-ready explanation: Mark-sweep first traces from GC roots and marks reachable objects, then sweeps memory to reclaim unmarked objects. Basic mark-sweep can leave fragmentation; collectors may therefore compact or use regions/generational strategies to improve allocation behavior.

Why / where it is used: The concept is foundational even though modern collectors use more advanced concurrent/parallel variants of tracing, evacuation and compaction.

- Mark: start from GC roots and mark reachable objects.
- Sweep: reclaim memory occupied by unmarked/unreachable objects.
- Problem: sweeping alone can leave fragmentation.
- Mark-compact adds compaction so live objects are moved together.

[VERY IMPORTANT] Q80. CMS vs G1 vs ZGC

Interview-ready explanation: CMS was a low-pause mostly-concurrent old-generation collector but suffered fragmentation and was removed from modern JDKs. G1 is a region-based balanced-throughput/latency collector and has been the default on server-class HotSpot for years. ZGC is a highly concurrent low-latency collector designed for very short pauses even with large heaps.

Why / where it is used: Choose based on SLA and workload: G1 is a strong general default; ZGC is attractive when tail latency/pause time matters most; CMS is historical knowledge rather than a modern deployment choice.

CMS	G1	ZGC
Legacy concurrent mark-sweep collector; removed from modern JDKs	Region-based collector; default in mainstream HotSpot since Java 9	Low-latency collector designed for very large heaps and very short pauses
Focused on reducing old-gen pauses but suffered fragmentation	Balances throughput and pause-time goals; performs young and mixed collections	Performs most work concurrently; modern ZGC supports generational operation

CMS	G1	ZGC
Do not choose for modern Java because it is removed	Strong general-purpose default	Choose when tail latency/very short pauses are a primary requirement

Q81. GC types/use cases and Eden memory

Interview-ready explanation: Generational GC relies on the observation that most objects die young. New objects are allocated in Eden, survivors may move through Survivor regions/spaces, and long-lived objects are eventually promoted to the old generation; young collections focus on reclaiming short-lived objects cheaply.

Why / where it is used: Understanding Eden and generations helps interpret GC logs, allocation-rate issues and why creating many temporary objects can increase young-GC frequency without necessarily being a memory leak.

Generational collectors exploit the observation that most objects die young. New allocations generally start in Eden (young generation), surviving objects may move through survivor regions/spaces and eventually be promoted to old generation.

- Young/Minor GC: collects young-generation objects.
- Mixed GC (G1): can collect young regions plus selected old regions.
- Full GC: broader stop-the-world collection; usually more expensive and something to investigate if frequent.
- Eden is where most new objects are allocated, often using thread-local allocation buffers (TLABs).
- G1 uses regions but still has logical Eden/Survivor/Old roles.

Q82. Java garbage collectors: when would you use which?

Interview-ready explanation: Collector choice is a trade-off among throughput, pause latency, heap size and operational simplicity. Serial is simple for tiny/single-core workloads; Parallel GC favors throughput; G1 balances throughput and predictable pauses; ZGC targets very low pause times; Shenandoah is another concurrent low-pause option in supported JDKs.

Why / where it is used: Start with a modern default unless measurements show an SLA problem. Tune only after observing real allocation, pause and CPU behavior.

Collector	Typical choice
Serial GC	Small heaps/simple applications where one GC thread is sufficient
Parallel GC	Throughput-oriented batch/CPU workloads where pauses are acceptable
G1	General server workloads needing balanced throughput and pause goals
ZGC	Large heaps and latency-sensitive systems requiring very small pauses

10. Modern Java: 8 / 11 / 17 / 21 / 25

Q83. Top 2 important features of Java 8, 11, 17, 21 and 25

Interview-ready explanation: Interview answers should focus on a few high-impact features rather than reciting release notes: Java 8 brought lambdas/Streams and CompletableFuture/default methods; Java 11 standardized the modern HTTP Client and added useful String/files APIs; Java 17 is an LTS with records/sealed classes/pattern matching progress; Java 21 is an LTS with virtual threads and finalized pattern matching for switch/record patterns; Java 25 is an LTS that continues language/runtime evolution.

Why / where it is used: In interviews, connect a feature to how you used it and mention migration concerns such as removed/deprecated APIs, module/access changes, dependency compatibility and performance testing.

Version	Two interview-relevant features
Java 8	Lambda expressions + functional interfaces; Stream API (plus Optional/default interface methods)
Java 11	Standard HTTP Client; useful String/Files API additions such as <code>isBlank()</code> , <code>lines()</code> , <code>repeat()</code> , <code>readString()/writeString()</code>
Java 17	Sealed classes; pattern matching for instanceof. Records are also widely used in Java 17 although finalized in Java 16.
Java 21	Virtual threads; pattern matching for switch + record patterns
Java 25	Scoped Values finalized; Module Import Declarations finalized. Compact Source Files + Instance Main Methods are also a Java 25 language feature.

NOTE: Java feature/version answers should distinguish “introduced/finalized in version X” from “commonly available in an LTS version.”

Q84. What is a Java record? Are records immutable?

Interview-ready explanation: A record is a compact syntax for a data carrier whose components generate final fields, accessors, constructor, equals/hashCode/toString. Records are shallowly immutable: their component references cannot be reassigned, but an object referenced by a component can still be mutable.

Why / where it is used: Use records for DTOs, messages, keys and value-like results where identity is the data itself and you do not need mutable bean semantics.

A record is a compact class for transparent data carriers. The compiler generates component accessors, `equals()`, `hashCode()`, `toString()` and a canonical constructor structure.

- Record components are final fields, so the record is shallowly immutable.
- If a component refers to a mutable object such as List, that object can still mutate unless you defensively copy it.
- Records are implicitly final and can implement interfaces.

```
record Employee(int id, String name) { }
```

Q85. What is a sealed class?

Interview-ready explanation: A sealed class/interface explicitly restricts which types are allowed to extend or implement it through permits (or same-module/package rules). Permitted subclasses must declare themselves final, sealed or non-sealed.

Why / where it is used: Sealed hierarchies are useful for closed domain models such as expression trees, result states or payment types and work especially well with exhaustive pattern matching.

A sealed class/interface restricts which classes/interfaces may directly extend or implement it.

- Permitted subclasses must explicitly be final, sealed, or non-sealed (subject to the language rules).
- Useful for controlled hierarchies and exhaustive pattern matching.

```
sealed interface Shape permits Circle, Rectangle { }  
final class Circle implements Shape { }  
non-sealed class Rectangle implements Shape { }
```

Q86. What is pattern matching in modern Java?

Interview-ready explanation: Pattern matching lets Java combine type testing and variable binding so code does not repeat casts. Modern Java supports pattern matching for instanceof and switch, including record patterns in appropriate versions.

Why / where it is used: It makes closed polymorphic domain handling shorter and safer, especially together with records and sealed types.

Pattern matching reduces boilerplate when checking a type and extracting its value. Modern Java includes pattern matching for instanceof, switch and record patterns.

```
if (obj instanceof String s) {  
    System.out.println(s.length());  
}  
  
String result = switch (shape) {  
    case Circle c -> "circle";  
    case Rectangle r -> "rectangle";  
};
```

[VERY IMPORTANT] Q87. What is a virtual thread?

Interview-ready explanation: A virtual thread is a lightweight `java.lang.Thread` scheduled by the JVM rather than permanently occupying one OS platform thread. Blocking a virtual thread can unmount it in many situations, allowing a small number of carrier threads to support very high concurrency.

Why / where it is used: Use virtual threads for request-per-thread workloads dominated by blocking I/O. They do not make CPU-bound work execute faster than available cores and you still need to control downstream concurrency such as database connection counts.

A virtual thread is a lightweight Thread managed by the JVM rather than being permanently mapped one-to-one to an OS thread. Java 21 finalized virtual threads.

- Excellent for high-concurrency workloads that spend much time blocking on I/O.
- They simplify the thread-per-request style without needing huge platform-thread pools.
- They do not make CPU-bound work faster; CPU parallelism is still limited by cores.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    executor.submit(() -> callRemoteService());  
}
```

11. Annotations

Q88. What are annotations? Can you create your own annotations?

Interview-ready explanation: Annotations attach structured metadata to classes, methods, fields, parameters and other program elements. Depending on retention, the compiler, annotation processors or runtime reflection/frameworks can inspect that metadata. Custom annotations are declared with `@interface`.

Why / where it is used: Frameworks such as Spring, JUnit and validation libraries use annotations to discover components, map requests, mark tests and define constraints without hard-coding every class.

Annotations attach metadata to code elements. The compiler, runtime reflection, annotation processors and frameworks can inspect that metadata. Yes, you create a custom annotation using `@interface`.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Audit {
    String value() default "";
}
```

Q89. What built-in annotations are provided by Java?

Interview-ready explanation: Common built-in annotations include `@Override`, `@Deprecated`, `@SuppressWarnings`, `@SafeVarargs` and `@FunctionalInterface`. Meta-annotations such as `@Retention`, `@Target`, `@Documented`, `@Inherited` and `@Repeatable` define how custom annotations behave.

Why / where it is used: Know the difference between compiler-checking annotations and metadata retained for runtime/framework use.

- `@Override` - verifies a method overrides/implements a supertype method.
- `@Deprecated` - marks APIs that should no longer be used.
- `@SuppressWarnings` - suppresses selected compiler warnings.
- `@SafeVarargs` - asserts certain generic varargs usage is safe.
- `@FunctionalInterface` - verifies a SAM/functional interface.
- Meta-annotations include `@Retention`, `@Target`, `@Documented`, `@Inherited` and `@Repeatable`.

12. Testing, Mockito & Dependency Injection

Q90. Difference between Unit Testing and Integration Testing

Interview-ready explanation: A unit test isolates one small unit of behavior and replaces external collaborators where appropriate, so it is fast and deterministic. An integration test verifies that multiple real components work together, such as service + database/repository + serialization or an HTTP boundary.

Why / where it is used: Healthy test suites use many focused unit tests plus fewer integration/end-to-end tests to catch wiring, configuration and infrastructure problems unit tests cannot see.

Unit test	Integration test
Tests one unit/class/function in isolation	Tests interaction between multiple components/systems
Dependencies often mocked/stubbed	Uses more real dependencies/infrastructure
Very fast, deterministic, many tests	Slower, fewer tests, catches wiring/contract issues
Example: service method with mocked repository	Example: service + real DB/test container + HTTP layer

Q91. How do you assert that code throws an exception in JUnit 5?

Interview-ready explanation: JUnit 5 `assertThrows()` executes a lambda and returns the thrown exception if it matches the expected type. You can then assert the message or other fields, which verifies both that failure occurs and that it is the intended failure.

Why / where it is used: This is cleaner and more precise than old expected-exception annotations because assertions can surround exactly the statement that should fail.

```
@Test
void rejectsNegativeAge() {
    IllegalArgumentException ex = assertThrows(
        IllegalArgumentException.class,
        () -> service.setAge(-1)
    );
    assertEquals("age must be positive", ex.getMessage());
}
```

Q92. @Mock vs @InjectMocks in Mockito

Interview-ready explanation: `@Mock` creates a Mockito mock for a dependency. `@InjectMocks` creates/initializes the system-under-test and asks Mockito to inject available mocks into it, preferring constructor injection and then other strategies as applicable.

Why / where it is used: Use mocks for collaborators you want to control/verify. Do not confuse `@InjectMocks` with a dependency-injection container; it is a Mockito convenience for unit tests.

@Mock	@InjectMocks
Creates a Mockito mock for a dependency	Creates the class under test and tries to inject available mocks/spies into it
No real behavior unless stubbed	The object itself is generally real; its dependencies may be mocks

@Mock	@InjectMocks
Example: Repository repository	Example: Service service receiving repository

Q93. Mock vs Spy

Interview-ready explanation: A mock is a fully simulated object whose unstubbed behavior returns Mockito defaults. A spy wraps a real object: real methods execute unless you stub them, so it is partial mocking.

Why / where it is used: Prefer mocks for external collaborators. Use spies sparingly when testing legacy code or when only a small part of a real object must be replaced, because real side effects can surprise the test.

Mock	Spy
Fully simulated object	Wraps/uses a real object by default
Methods return Mockito defaults unless stubbed	Real methods execute unless stubbed
Best for dependencies you want isolated	Useful for partial mocking, but use sparingly
when(mock.m()).thenReturn(x)	doReturn(x).when(spy).m() is often safer

Q94. doReturn() vs thenReturn()

Interview-ready explanation: when(x.method()).thenReturn(v) evaluates the method call during stubbing. For a pure mock that is normally harmless; for a spy it may invoke real code. doReturn(v).when(x).method() installs the stub without calling the real method first.

Why / where it is used: Use doReturn-style stubbing for spies or methods where evaluating the real call during setup would be unsafe/expensive.

Both can stub non-void methods, but the important difference appears with spies. when(spy.realMethod()).thenReturn(x) calls the real method during stubbing; doReturn(x).when(spy).realMethod() does not.

```
doReturn("stubbed").when(spy).load();
when(mock.load()).thenReturn("stubbed");
```

Q95. How do you mock a void method?

Interview-ready explanation: Void methods are stubbed with Mockito's doNothing(), doThrow(), doAnswer() family because there is no return value for thenReturn(). Often the best test is not to 'mock the void' but to verify that the expected interaction occurred with the right arguments.

Why / where it is used: This is common for repository.save(), notification.send() and audit/log side-effect dependencies.

- Void methods are commonly tested by verifying interaction or state change rather than returning a value.

```
doNothing().when(mock).save(any());
doThrow(new RuntimeException("boom")).when(mock).delete(any());
doAnswer(inv -> { /* custom behavior */ return null; }).when(mock).send(any());

verify(mock).save(any());
```

Q96. Constructor injection vs setter injection

Interview-ready explanation: Constructor injection provides all required dependencies when the object is created, which makes invalid partially initialized objects harder to create and allows fields to be final. Setter injection leaves a dependency replaceable/optional after construction.

Why / where it is used: Prefer constructor injection for required collaborators and testability. Use setter injection only when a dependency is genuinely optional or reconfigurable.

Constructor injection	Setter injection
Best for required dependencies	Useful for optional/reconfigurable dependencies
Object is valid immediately after construction	Object may exist temporarily incomplete
Supports final fields/immutability	Dependencies usually remain mutable
Easy to test explicitly	Can hide required dependencies

INTERVIEW TIP: For required dependencies, constructor injection is generally preferred.

13. Reactive Programming

Q97. Reactive programming: Mono and Flux with non-blocking I/O

Interview-ready explanation: Reactive programming models asynchronous streams of data and propagates demand/backpressure through a pipeline. In Project Reactor, `Mono<T>` represents zero or one item and `Flux<T>` zero to many items; operators build a lazy pipeline that runs when subscribed.

Why / where it is used: It works best when the full I/O chain is non-blocking, such as Spring WebFlux with reactive drivers. Wrapping blocking JDBC or file calls in `Mono/Flux` does not automatically make them non-blocking and can still consume scarce threads.

Project Reactor is a reactive library commonly used by Spring WebFlux. `Mono<T>` represents 0 or 1 asynchronous element; `Flux<T>` represents 0 to many elements.

- Reactive streams support asynchronous pipelines and backpressure.
- They work well with non-blocking I/O when the entire call chain is non-blocking.
- Putting a blocking JDBC/file/network call inside a `Mono/Flux` does not magically make that operation non-blocking.
- Common operators: `map`, `flatMap`, `filter`, `zip`, `onErrorResume`, `subscribe`.

```
Mono<User> oneUser = userClient.findById(id);  
Flux<User> manyUsers = userClient.findAll();
```

14. Stream API Coding Questions

The original question bank already contained several answers. They are reorganized here and normalized into interview-ready Java code.

Q98. Count every character in a String using Stream API

Interview-ready explanation: The solution converts each character to an object stream and groups by character using `Collectors.counting()`. `groupingBy` builds a map from character to occurrence count.

Why / where it is used: Complexity is $O(n)$ over the number of characters, with extra map space for distinct characters. If Unicode code points beyond `char` are important, use `codePoints()` rather than `chars()`.

```
String str = "abcdabghsk";
Map<Character, Long> counts = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
System.out.println(counts);
```

Q99. Merge two lists using Stream API

Interview-ready explanation: `Stream.concat()` produces one stream that first emits `list1` then `list2`. `flatMap` over `Stream.of(list1, list2)` is an equivalent pattern when you may have several nested lists.

Why / where it is used: The result preserves encounter order for ordered input streams. In modern Java, `toList()` returns an unmodifiable list implementation, so use `Collectors.toCollection(ArrayList::new)` if you need a mutable result.

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4);
List<Integer> list2 = Arrays.asList(5, 6, 7, 8);

List<Integer> merged = Stream.concat(list1.stream(), list2.stream()).toList();
// Alternative: Stream.of(list1, list2).flatMap(List::stream).toList();
```

Q100. Reverse a String using Stream API

Interview-ready explanation: The `IntStream` indexes from 0 to `length-1` and reads characters from the opposite end, then joins them. It demonstrates Stream API mechanics, although new `StringBuilder(str).reverse()` is simpler and faster for production code.

Why / where it is used: Be aware that `char`-based reversal can split Unicode surrogate pairs; code-point-aware reversal is needed for full Unicode correctness.

```
String str = "Stream";
String reversed = IntStream.range(0, str.length())
    .mapToObj(i -> String.valueOf(str.charAt(str.length() - 1 - i)))
    .collect(Collectors.joining());
System.out.println(reversed); // maertS
```

Q101. Delete all occurrences of 'c' from a String

Interview-ready explanation: The stream filters out every character equal to `'c'` and joins the remaining characters back into a `String`. The original `String` is unchanged because `String` is immutable.

Why / where it is used: This is an $O(n)$ filter operation and is a good example of `chars()` + `mapToObj()` + `Collectors.joining()`.

```
String s = "occurance";
String ans = s.chars()
    .mapToObj(c -> (char) c)
    .filter(ch -> ch != 'c')
    .map(String::valueOf)
    .collect(Collectors.joining());
System.out.println(ans); // ourane
```

Q102. List duplicate characters in a String

Interview-ready explanation: First count occurrences, then stream the map entries and keep only characters whose count is greater than one. Mapping the surviving entries back to their key gives the duplicate characters.

Why / where it is used: If deterministic encounter order matters, collect counts into a LinkedHashMap; a plain groupingBy HashMap does not promise display order.

```
String s = "programming";
List<Character> duplicates = s.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .toList();
System.out.println(duplicates);
```

Q103. Count how many times each word appears in a List<String>

Interview-ready explanation: groupingBy(Function.identity(), counting()) groups equal words and counts how many elements fall into each group. It is the standard Stream pattern for frequency maps.

Why / where it is used: If case should be ignored or punctuation normalized, preprocess each word with map() before grouping.

```
List<String> words = Arrays.asList("apple", "banana", "apple", "orange", "banana");
Map<String, Long> wordCount = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
// apple=2, banana=2, orange=1
```

Q104. Return the second-highest distinct word length in a sentence

Interview-ready explanation: Map each word to its length, remove duplicate lengths, sort descending, skip the largest and take the next. distinct() is important because the question asks for the second-highest distinct length, not the length of the second word after sorting.

Why / where it is used: For huge data sets, a one-pass top-two algorithm would be more efficient than sorting, but this Stream answer is concise and interview-friendly.

```
String sentence = "I am working in motivated organization";
int result = Arrays.stream(sentence.split("\\s+"))
    .map(String::length)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(0);
System.out.println(result); // 9
```


Q105. Find even integers from a mixed List<String>

Interview-ready explanation: Filter to strings that represent integers, parse them, then keep values divisible by two. Validating before parse avoids NumberFormatException for inputs such as 'one' and 'ab12'.

Why / where it is used: If signed integers are allowed, use a pattern that accepts '-' or use a parse helper with try/catch. Be explicit about null handling.

```
List<String> list = Arrays.asList("one", "5", "ab12", "abc", "8", "20");
List<Integer> even = list.stream()
    .filter(Objects::nonNull)
    .filter(s -> s.matches("-?\\d+"))
    .map(Integer::parseInt)
    .filter(n -> n % 2 == 0)
    .toList();
System.out.println(even); // [8, 20]
```

Q106. Names ending with "lal" using Stream API

Interview-ready explanation: Use filter(name -> name.endsWith("lal")) to retain only matching names. endsWith is case-sensitive, so clarify whether names such as 'LAL' should count.

Why / where it is used: This is a direct Predicate/filter question; no mutation of the original list occurs.

```
List<String> list = Arrays.asList("Ram", "Vinitha", "Mohanlal", "Jawaharlal");
List<String> result = list.stream()
    .filter(name -> name.endsWith("lal"))
    .toList();
```

Q107. Names NOT ending with "lal" using Stream API

Interview-ready explanation: Negate the predicate: filter(name -> !name.endsWith("lal")). The stream returns a new result and leaves the source list unchanged.

Why / where it is used: If nulls are possible, handle them before calling endsWith as shown in the next question.

```
List<String> result = list.stream()
    .filter(name -> !name.endsWith("lal"))
    .toList();
```

Q108. If the names list contains null, avoid NullPointerException and return names not ending with "lal"

Interview-ready explanation: Filter Objects::nonNull first, then apply the suffix predicate. Stream filters run left-to-right, so null elements are removed before endsWith is invoked.

Why / where it is used: This demonstrates safe predicate ordering and is preferable to catching NullPointerException.

```
List<String> result = list.stream()
    .filter(Objects::nonNull)
    .filter(name -> !name.endsWith("lal"))
    .toList();
```

Q109. Find the second-largest number using Stream API

Interview-ready explanation: Remove duplicates, sort descending, skip the maximum and take the next value. `distinct()` is required if repeated copies of the largest value should not make that same value count as 'second largest'.

Why / where it is used: Define behavior for fewer than two distinct values; `orElse(null)`, `Optional<Integer>` or an exception are all possible contracts.

```
List<Integer> numbers = Arrays.asList(10, 5, 15, 20, 8);
Integer secondLargest = numbers.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(null); // 15
```

Q110. Find unique characters of a String using Stream API

Interview-ready explanation: `chars()` creates an `IntStream` of UTF-16 code units, `mapToObj` converts them to `Character`, and collecting to a `Set` removes duplicates.

Why / where it is used: Use `LinkedHashSet` when you want first-encounter order, and use `codePoints()` rather than `chars()` for full Unicode code points.

```
String input = "hello world";
Set<Character> uniqueChars = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.toSet());
```

NOTE: `Collectors.toSet()` does not guarantee encounter order. Use `LinkedHashSet::new` if insertion order matters.

Q111. Convert List<Employee> to Map<empId, empName>

Interview-ready explanation: `Collectors.toMap()` maps each `Employee` to a key from `getEmpId` and a value from `getEmpName`. Employee IDs must be unique unless you provide a merge function.

Why / where it is used: This pattern is common when converting a list returned from a database/API into a fast lookup map.

```
Map<Integer, String> employeeMap = employees.stream()
    .collect(Collectors.toMap(Employee::getEmpId, Employee::getEmpName));
```

NOTE: If duplicate `empId` values are possible, provide a merge function to avoid `IllegalStateException`.

Q112. Convert List<Employee> to Map<empId, empName> while maintaining insertion order

Interview-ready explanation: The four-argument `toMap` overload lets you provide both a duplicate-key merge function and a map supplier. Supplying `LinkedHashMap::new` preserves insertion order from the stream.

Why / where it is used: This is the right choice when output order is part of the API/UI requirement.

```
Map<Integer, String> employeeMap = employees.stream()
    .collect(Collectors.toMap(
        Employee::getEmpId,
        Employee::getEmpName,
        (v1, v2) -> v1,
        LinkedHashMap::new
    ));
```

Q113. Find the vehicle that travelled the maximum total distance in August

Interview-ready explanation: Filter journeys to August, group by vehicleId and sum distance, choose the entry with the largest total, then resolve that ID to a vehicle name. The pipeline mirrors a SQL GROUP BY + SUM + MAX + join.

Why / where it is used: If data spans multiple years, include the year filter. For large data, build a vehicleId->Vehicle map first rather than repeatedly scanning vehicleList.

```
String maxDistanceVehicle = journeyList.stream()
    .filter(j -> j.getDateOfJourney().getMonthValue() == 8)
    .collect(Collectors.groupingBy(
        Journey::getVehicleId,
        Collectors.summingDouble(Journey::getDistanceTravelled)))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .flatMap(e -> vehicleList.stream()
        .filter(v -> Objects.equals(v.getId(), e.getKey()))
        .findFirst())
    .map(Vehicle::getName)
    .orElse("No vehicles travelled in August");
```

NOTE: If multiple years are present, also filter by year; “August” alone combines all August journeys across years.

Q114. Return duplicated integers from an ArrayList<Integer>

Interview-ready explanation: Build a frequency map and return keys whose count is greater than one. This is O(n) expected time rather than the nested-stream O(n²) approach often shown in quick solutions.

Why / where it is used: If output order should follow first occurrence, collect the frequency map into LinkedHashMap.

```
List<Integer> nums = Arrays.asList(1, 2, 3, 2, 4, 5, 3, 6);
List<Integer> duplicates = nums.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .toList(); // [2, 3] (order depends on map implementation)
```

NOTE: This avoids nested streams and avoids relying on Integer reference comparison with ==.

Q115. Return employees whose name starts with 'A' and city is Mumbai or Pune

Interview-ready explanation: Compose two predicates: name starts with A, and city equals Mumbai or Pune ignoring case. Predicate.and() makes the business rule readable and reusable.

Why / where it is used: In production, null-safe access may be required if Employee name/city can be null.

```
Predicate<Employee> nameStartsWithA = e -> e.getName().startsWith("A");
Predicate<Employee> cityIsMumbaiOrPune = e ->
    e.getCity().equalsIgnoreCase("Mumbai") || e.getCity().equalsIgnoreCase("Pune");

List<Employee> result = employees.stream()
    .filter(nameStartsWithA.and(cityIsMumbaiOrPune))
    .toList();
```

Final 2-Minute Revision Sheet

Topic	One-line answer
HashMap	hash -> bucket -> equals; default capacity 16, load factor 0.75; collisions may treeify.
ConcurrentHashMap	Java 8+ CAS + fine-grained bin synchronization; no null keys/values; weakly consistent iterators.
Comparable vs Comparator	compareTo = natural order; compare = external/custom order.
String / Builder / Buffer	String immutable; Builder mutable fastest; Buffer mutable synchronized.
Functional interface	Exactly one abstract method; lambda supplies its implementation.
PECS	Producer Extends, Consumer Super.
Exception override	Cannot broaden checked exception.
volatile vs Atomic	volatile visibility/order; Atomic gives atomic compound operations via CAS-style APIs.
wait vs sleep	wait releases monitor and needs ownership; sleep does not release monitor.
Future vs CompletableFuture	Future is basic result; CompletableFuture supports composition and callbacks.
JMM	Visibility + ordering + atomicity rules via happens-before.
GC	G1 general default; ZGC low latency; CMS is removed.
Record	Compact data carrier; shallowly immutable.
Virtual thread	Lightweight Java 21 thread, excellent for high-concurrency blocking I/O.
Mockito spy	Real methods by default; doReturn is safer when stubbing a spy.

15. Additional OOP & Constructor Questions from Your Notes

These additional questions were supplied during the review process. They are included because they are common Java interview topics and match the same answer standard as the main question bank.

Q116. What is abstraction? How do you achieve abstraction in Java?

Interview-ready explanation: Abstraction is the process of hiding implementation details and exposing only the essential behavior a caller needs. It focuses on what an object can do rather than how it does it. In Java, abstraction is mainly expressed through abstract classes and interfaces.

Why / where it is used: Use abstraction to keep callers independent of implementation details. For example, business code can depend on a `PaymentGateway` interface while `StripePaymentGateway` or `PayPalPaymentGateway` contains the real network logic. This makes implementations replaceable and easier to test.

Q117. Can an abstract class have a constructor? Why?

Interview-ready explanation: Yes. An abstract class cannot be instantiated directly, but its constructor still runs whenever a concrete subclass object is created. The constructor initializes the inherited state owned by the abstract base class.

Why / where it is used: This is useful when every subclass shares common fields or validation. A subclass constructor implicitly or explicitly calls `super(...)`, so the base portion of the object is initialized before the subclass-specific portion.

Q118. What is encapsulation? Why is it important?

Interview-ready explanation: Encapsulation bundles data and the operations that protect that data inside a class, while restricting direct access to internal state. In Java this is commonly achieved with private fields and public/protected methods that enforce rules.

Why / where it is used: It prevents callers from putting an object into an invalid state and reduces coupling. For example, a `BankAccount` should expose `deposit()/withdraw()` rather than a public `balance` field so it can validate amounts and preserve invariants.

Q119. What is inheritance? What types of inheritance does Java support?

Interview-ready explanation: Inheritance lets a subclass reuse and specialize behavior from a superclass, modeling an is-a relationship. Java supports single, multilevel and hierarchical class inheritance. Java does not support multiple inheritance of classes, but a class can implement multiple interfaces.

Why / where it is used: Use inheritance only when the child truly satisfies the parent's contract. For code reuse without a true subtype relationship, composition is usually safer and more flexible.

Q120. What is polymorphism? Explain compile-time vs runtime polymorphism.

Interview-ready explanation: Polymorphism means one interface/name can represent different behaviors. Compile-time polymorphism is typically method overloading, where the compiler selects a method from the parameter list. Runtime polymorphism is method overriding, where dynamic dispatch selects the implementation based on the actual object.

Why / where it is used: Runtime polymorphism is fundamental to interfaces, dependency injection and extensible frameworks: code can work with a `List`, `PaymentService` or `Shape` reference without knowing the concrete implementation.

Q121. What are SOLID principles?

Interview-ready explanation: SOLID is a set of five object-oriented design principles for maintainable and flexible software: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion.

Why / where it is used: SRP: one reason to change. OCP: extend behavior without modifying stable code. LSP: subtypes must honor the parent contract. ISP: prefer small focused interfaces over large 'fat' interfaces. DIP: high-level policy should depend on abstractions, not concrete low-level details. These principles guide service boundaries, interfaces and dependency injection.

Q122. What are the types of constructors in Java?

Interview-ready explanation: The two fundamental categories are no-argument and parameterized constructors. If you declare no constructor at all, the compiler supplies an implicit default no-argument constructor. You can also write an explicit no-arg constructor, parameterized constructors and common design patterns such as copy constructors or private constructors.

Why / where it is used: Use an explicit no-arg constructor when frameworks or defaults require it, parameterized constructors to create valid fully initialized objects, copy constructors to copy state intentionally, and private constructors to control object creation in factories/singletons/utility classes.

Type	Who creates it?	Main use
Implicit default constructor	Compiler	Only when no constructor is declared
Explicit no-arg	Programmer	Custom default initialization / frameworks
Parameterized	Programmer	Initialize each object with supplied state
Copy constructor pattern	Programmer	Create new object from existing object
Private constructor pattern	Programmer	Control/prevent external construction

Q123. What is constructor chaining? Explain this() vs super().

Interview-ready explanation: Constructor chaining means one constructor delegates to another constructor. `this(...)` calls another constructor in the same class; `super(...)` calls a constructor of the immediate parent. Either call, when present, must be the first statement in the constructor, and you cannot call both directly from the same constructor.

Why / where it is used: Use `this(...)` to centralize overloaded-constructor logic and remove duplication. Use `super(...)` to initialize the inherited parent portion of a subclass. If no constructor call is written, the compiler inserts `super()` when a compatible no-arg parent constructor exists.

Feature	this()	super()
Target	Another constructor in same class	Constructor in immediate parent
Use	Constructor overloading/delegation	Inheritance initialization
Compiler inserts automatically?	No	Yes: <code>super()</code> if omitted and valid
Position	Must be first statement	Must be first statement

Q124. Can constructors be private, final, static or abstract?

Interview-ready explanation: A constructor can be private, which restricts creation to code inside the class. Constructors cannot be final because they are never inherited/overridden, cannot be static because their purpose is to initialize an instance, and cannot be abstract because they must contain executable initialization logic and are not inherited.

Why / where it is used: Private constructors are common in Singletons, factories and utility classes. For class-level initialization use a static initializer; for enforced subclass behavior use abstract methods, not abstract constructors.

Modifier	Allowed?	Reason
private	Yes	Controls object creation
final	No	Constructors are not inherited/overridden
static	No	Constructors initialize an instance
abstract	No	Constructor must have executable body and is not inherited

Q125. What is a copy constructor in Java? Does Java provide one by default?

Interview-ready explanation: A copy constructor is a programmer-defined constructor that accepts another object of the same class and copies its state into a new object. Java does not automatically generate a copy constructor like C++; you must define the copy semantics yourself.

Why / where it is used: A copy constructor is often clearer than clone() because you decide explicitly whether each field is copied shallowly or deeply. For mutable nested state, copy the nested object too if the new instance must be independent.

```
class Employee {
    private final int id;
    private final String name;

    Employee(Employee other) {
        this.id = other.id;
        this.name = other.name;
    }
}
```

Q126. How does String concatenation work internally? What happens when + is used inside a loop?

Interview-ready explanation: Because String is immutable, a concatenation result is a new String value rather than an in-place modification. Modern compilers/runtime use optimized concatenation strategies (historically StringBuilder-style lowering; modern JDKs may use invokedynamic/StringConcatFactory), but repeated growth in a loop still repeatedly builds larger results.

Why / where it is used: For repeated concatenation, use one StringBuilder and append to it. That avoids creating/copying a growing chain of intermediate String values and usually reduces time complexity, allocation rate and garbage-collection pressure.

```
StringBuilder sb = new StringBuilder();
for (String part : parts) {
    sb.append(part);
}
String result = sb.toString();
```

Source basis: Java_Interview_Question_Bank_Topic_Wise.pdf plus the additional answer material and questions supplied in this chat. Existing short-answer content is preserved as quick-revision material, with detailed interview-ready context added above it.